

INTRODUÇÃO À PROGRAMAÇÃO EM LÓGICA COM PROLOG

SWI-Prolog online: <https://swish.swi-prolog.org/>

1. FATOS

Em Prolog (*PROgramming in LOGic*), problemas são representados através de declarações lógicas, de forma que o motor de inferência usa um provador de teoremas baseado no Princípio da Resolução para encontrar as soluções.

Portanto, um programa em Prolog é essencialmente uma Base de Conhecimento, onde as entidades mais básicas são os Fatos, que declaram verdades absolutas e incondicionais sobre o domínio do problema.

Sintaticamente, um fato é uma relação (predicado) aplicada a um ou mais argumentos.

Regras de Sintaxe:

- Constantes e predicados começam com letra minúscula;
- Variáveis começam com letra maiúscula ou underscore (_);
- Toda declaração em Prolog deve terminar com um ponto final (.).

Exemplo: Onde em Lógica de Primeira Ordem escreve-se $\text{Pai}(x, \text{maria})$, em Prolog escreve-se `pai(X, maria)`.

Exemplo: Um ecossistema simples:

```
% Fatos: O que animais comem.  
come(urso, peixe).  
come(urso, raposa).  
come(veado, grama).  
come(peixe, alga).  
come(raposa, coelho).  
come(coelho, grama).
```

```
% Fatos: Classificação de seres vivos.  
animal(urso).  
animal(raposa).  
animal(veado).  
animal(coelho).  
animal(peixe).  
planta(grama).  
planta(alga).
```

No console do Prolog, são feitas Consultas (*queries*) da seguinte forma:

?- come(urso, peixe).

O motor de inferência tenta unificar a consulta com um dos fatos da base de conhecimento. Se ele consegue, como neste exemplo onde come(urso, peixe) é um fato, responde true. Caso contrário, responde false.

Se na consulta forem usadas variáveis, equivale a perguntar: “Existe algum X tal que...?” Neste caso, as respostas são os valores que tornam a consulta verdadeira.

?- come(urso, X).

“Existe algum X tal que urso come X?”

Respostas: X = peixe ;
X = raposa.

Obs.: O Prolog encontra a primeira solução. Pressionando a tecla ‘;’ na linha de comando ou “Next” na versão online ele busca a próxima solução.

2. REGRAS

Enquanto fatos são verdades incondicionais, Regras são verdades condicionais. Uma regra diz que um fato é verdadeiro se uma série de outras condições também forem verdadeiras.

Regras representam implicações lógicas, mais precisamente, Cláusulas Definidas. Uma Cláusula Definida é uma disjunção de literais em que exatamente um deles é positivo.

Cláusulas de Horn: $\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_{n-1} \vee P_n$

$\neg(P_1 \wedge P_2 \wedge \dots \wedge P_{n-1}) \vee P_n$

$P_1 \wedge P_2 \wedge \dots \wedge P_{n-1} \rightarrow P_n$

De Morgan

Equivalência de Implicação

Sintaxe da Regra:

Cabeça :- Corpo₁, Corpo₂, ..., Corpo_n .

- A Cabeça é a conclusão.
- O símbolo ‘:-’ significa “se” (uma implicação da direita para a esquerda: $\leftarrow$).
- A vírgula (,) no corpo significa conjunção lógica ($\wedge$).
- O ponto e vírgula (;) significa disjunção lógica ($\vee$).
- A regra termina com um ponto final (.).

Exemplo: A seguinte fórmula em Lógica de Primeira Ordem:

$$\forall x \forall y (Come(x, y) \wedge Planta(y) \rightarrow Herbivoro(x))$$

Em Prolog fica: `herbivoro(X) :- come(X, Y), planta(Y).`

Obs.: Em Prolog, as variáveis na consulta estão quantificadas existencialmente, enquanto que variáveis em fatos e regras estão quantificadas universalmente.

Exemplo: Considere os fatos do ecossistema simples definidos anteriormente.

```
% Um animal X é herbívoro se ele come algo (Y) e esse algo é uma planta.  
herbivoro(X) :- come(X, Y), planta(Y).
```

```
% Um animal X é carnívoro se ele come algo (Y) e esse algo é um animal.  
carnivoro(X) :- come(X, Y), animal(Y).
```

```
% Dois animais X e Y são competidores se eles se alimentam da mesma coisa.  
competidores(X, Y) :-  
    come(X, Z),  
    come(Y, Z),  
    X \= Y.
```

O operador '`\=`' verifica se X não unifica com Y, ou seja, se são diferentes.

```
% Um predador de topo X é carnívoro e não é comido por nenhum outro animal.  
predador_topo(X) :- carnivoro(X), \+ come(_, X).
```

O símbolo '`_`' é a variável anônima e significa que não importa o valor.

O símbolo '`\+`' significa Negação por Falha: se em algum ramo da árvore de dedução a partir daquele ponto houver falha, a busca por uma resposta termina e o motor de inferência retorna false.

Neste exemplo, se for encontrado um único animal que coma X, então X não é um predador de topo.

3. UNIFICAÇÃO E BACKTRACKING

Em Prolog, uma consulta realiza essencialmente uma Busca em Profundidade em uma árvore de prova lógica.

Quando é feita uma consulta, o motor de inferência segue os seguintes passos:

1. Unificação: Ele tenta unificar a consulta (*query*) com um fato ou com a cabeça de uma regra. As variáveis recebem valores para que os dois termos fiquem idênticos;
2. Resolução: Se houve unificação com uma regra, ele tenta provar cada literal do corpo, da esquerda para a direita;
3. Backtracking (Retrocesso): Se a prova de um literal falha, volta-se até o último ponto de escolha onde havia mais de uma opção e tenta-se outra alternativa;
4. Quando houver unificação com um fato, o motor de inferência responde true se a consulta não contiver variáveis, ou responde com as substituições feitas que tornaram possível a unificação. Caso não haja unificação com nenhum fato, o motor de inferência responde false.

Exemplo: Considere a seguinte Base de Conhecimento que representa um grafo direcionado, onde aresta(X, Y) significa que existe no grafo uma aresta partindo de X e indo até Y.

```
aresta(a, b).  
aresta(b, c).  
aresta(c, d).  
aresta(a, e).
```

Predicado recursivo caminho(X, Y) que verifica se existe um caminho de X até Y:

% Caso Base: Existe caminho de X para Y se há uma aresta direta de X para Y.

```
caminho(X, Y) :- aresta(X, Y).
```

% Passo Recursivo: Existe caminho de X para Y se há uma aresta de X para um % nó intermediário Z, e um caminho de Z até Y.

```
caminho(X, Y) :- aresta(X, Z), caminho(Z, Y).
```

Consulta: ?- caminho(a, d).

- Falha no caso base (não há aresta(a, d));
- Tenta a regra recursiva:
 - aresta(a, Z): unifica $Z = b$ com sucesso, pois aresta(a, b) é um fato;
 - caminho(b, d):
 - Falha no caso base (não há aresta(b, d)).
 - Tenta a regra recursiva:
 - aresta(b, Z'): unifica $Z' = c$ com sucesso, pois aresta(b, c) é um fato;
 - caminho(c, d):
 - Caso base: sucesso, pois aresta(c, d) é um fato, retorna true.

Exemplo: Considere a seguinte Base de Conhecimento que representa a relação de paternidade, onde pai(X, Y) afirma que X é pai de Y.

pai(jinpachi, heihachi).
pai(heihachi, kazuya).
pai(kazuya, jin).

Predicado ancestral(X, Y) que diz que X é ancestral de Y se X for pai de Y, ou se houver uma cadeia de pais entre eles:

ancestral(X, Y) :- pai(X, Y).
ancestral(X, Y) :- pai(Z, Y), ancestral(X, Z).

Obs.: Sempre deve-se colocar as chamadas não-recursivas antes da chamada recursiva, pois isso evita que se entre em recursões muito grandes (ou até infinitas) sem necessidade. No exemplo anterior, não haveria razão de procurar os ancestrais de um dado Z caso ele não seja pai de Y.

4. CÁLCULOS EM PROLOG

Prolog não é uma linguagem adequada para se fazer cálculos complexos, no entanto é possível resolver expressões aritméticas simples.

Em Prolog, o operador '=' significa unificação, não atribuição de valor.

Exemplo: Se a consulta for: ?- X = 2 + 3.

A resposta será: X = 2 + 3. (a variável X armazena a equação literal)

5. LISTAS

Estruturas que armazenam dados em sequência são importantes em qualquer linguagem, uma vez que permitem que tais dados sejam processados de forma iterativa ou recursiva. Em Prolog, existem estruturas chamadas Listas que tem essa finalidade.

Como não existem laços de repetição em Prolog, Listas de dados são processadas por indução usando recursividade. A ideia é que toda lista não vazia é formada por um primeiro elemento seguido pelo restante, que também é uma Lista.

Formalmente, Listas em Prolog são definidas assim:

- Lista vazia: []
- Lista não-vazia: [H | T]

Onde H é o primeiro elemento, chamado de cabeça (*head*) e T é a Lista com os elementos restantes, chamada de cauda (*tail*).

Exemplo: Na lista [1, 2, 3], a cabeça H é 1 e a cauda T é [2, 3].

Exemplo: Predicado membro(E, L) que verifica se o elemento E pertence à lista L:

```
% Caso Base: X é membro da lista se X for o primeiro elemento (a cabeça).  
membro(X, [X | _]).      % não importa o que é a cauda
```

```
% Passo Recursivo: X é membro da lista [H | T] se ele for membro da cauda T.  
membro(X, [_ | T]) :- membro(X, T).  % já sei que não é a cabeça
```

Exemplo: Predicado concatena(L1, L2, R) que unifica em R a concatenação das listas L1 e L2:

```
% Caso Base: a primeira lista é vazia.  
concatena([], L2, L2).
```

```
% Passo Recursivo: a concatenação da lista [H | T] com L2 é a lista formada por  
% H e a concatenação de T com L2.  
concatena([H | T], L2, R) :-  
    concatena(T, L2, Rt),      % faz a concatenação de T com L2  
    R = [H | Rt].              % depois coloca H na frente
```

Obs.: A unificação do resultado poderia ser feita na cabeça da regra:
concatena([H | T], L2, [H | Rt]) :- concatena(T, L2, Rt).

Obs.: `append(L1, L2, R)` é um predicado nativo do Prolog usado unificar em R a concatenação das listas L1 e L2.

Exemplo: Predicado `comprimento(L, N)` que calcula o número de elementos N de uma lista L:

% Caso Base: o comprimento de uma lista vazia é 0.
`comprimento([ ], 0).`

% Passo Recursivo: o comprimento de uma lista `[H | T]` é 1 + comprimento de T.
`comprimento([ _ | T], N) :-` % não importa o que é a cabeça
 `comprimento(T, Nt),` % calcula o comprimento da cauda, Nt
 `N is Nt + 1.` % depois soma 1

Obs.: `length(L, N)` é um predicado nativo do Prolog usado para calcular o número de elementos N de uma lista L.